

Welcome to

COMP1870: Theoretical Foundations of Computer Science I

Contact details

Module leader: Dillon Mayhew

- ▶ Email: d.mayhew@leeds.ac.uk
- ▶ Teams Chat: Dillon Mayhew
- ▶ Student hour: Tuesdays, 1.00pm – 2.00pm,
Bragg Building foyer
- ▶ Office: Chemistry 3.23 (cross the second floor bridge from
the William Henry Bragg building and follow the signs)

Sir William Henry Bragg Building



Instructor-led sessions

- ▶ Lectures (first semester):
 - ▶ Tuesdays 3.00pm – 5.00pm,
Mechanical Engineering LT A
 - ▶ Fridays 9.00am – 11.00am,
Chemical and Process Engineering LT A
- ▶ Tutorials:
 - ▶ Your tutorial times are in your personalised timetable (two per week).
 - ▶ You will be able to attend other tutorial times, subject to room capacity.
 - ▶ Tutorials on Wednesday or Thursday covers the Tuesday lecture.
 - ▶ Tutorials on Monday or Tuesday covers the Friday lecture.
 - ▶ Tutorials start on Wednesday of Week 1.

Minerva page for COMP1870

On the Minerva page you will find:

- ▶ lecture notes,
- ▶ draft lecture slides (posted before the lecture),
- ▶ annotated lecture slides (posted after the lecture),
- ▶ weekly worksheets,
- ▶ submission links for portfolio exercises,
- ▶ links to lecture recordings,
- ▶ module announcements
(also sent by email - make sure you read all announcements).

Weekly worksheets

Worksheets are posted each Monday.

They contain a mixture of:

- ▶ tutorial exercises
(attempt these before your tutorial session, your tutor will go through solutions),
- ▶ portfolio exercises
(marked with a red border, you submit these as part of your assessment)
- ▶ advanced exercises
(marked with a blue star, intended to extend you)
- ▶ exam style exercises
(marked with a blue 'E', you will see questions like these in exams, but you should look at **all** exercises when preparing).

Portfolio submissions

- ▶ Each week the worksheet will contain 1-3 portfolio exercises (marked with a red border).
- ▶ Portfolio exercises are submitted in Gradescope (link is in the Minerva page).
- ▶ Submit by uploading an image file or pdf (there is an optional template for your submissions, but you can write your solutions on paper and photograph them).
- ▶ The passing mark is 40.
- ▶ You will receive feedback after your first submission.
- ▶ Exercises can be resubmitted after receiving feedback.
- ▶ Continue resubmitting until you receive a passing mark.
- ▶ You need to pass **every** portfolio submission to pass the portfolio.

Portfolio deadlines

- ▶ The due date for each portfolio submission is ten days after they are posted (posted Monday, due on Friday).
- ▶ There is an expectation that everyone will submit their portfolios in a timely fashion and stay actively engaged with the worksheets.
- ▶ You should aim to make your first submission before the due date. All submissions must be complete by the end of the semester.
- ▶ It is important to stay current with the worksheets — delaying your work to the end of the semester makes exam preparation much harder.

Assessment

Other assessment items:

- ▶ 2-hour Semester 1 exam in January
(covers Semester 1 material).
- ▶ 3-hour Semester 2 exam in May/June
(roughly $\frac{2}{3}$ Semester 2 material, $\frac{1}{3}$ Semester 1 material).

Break-down of final grade:

- ▶ Passing the portfolio guarantees a grade of at least 40.
- ▶ Assuming you pass the portfolio, your final grade will be
maximum of: 40 and $\left(\frac{1}{3}\text{January mark} + \frac{2}{3}\text{May/June mark}\right)$
- ▶ If you fail the portfolio, you will need to resit and your
mark will be capped at 40.

Online Q&A board

During lectures, post your questions on the Q&A board:
Go to [vevox.app](https://vevox.com) and enter ID 107-018-952, or use QR code



- ▶ Questions are posted anonymously and moderated.
- ▶ Any questions not answered during the lectures will be answered in the module Teams chat.

When you have questions

Everything we have talked about is in the **module handbook** (posted in Minerva).

Where to find help:

- ▶ ask your lecturer,
- ▶ ask your tutor,
- ▶ ask a question in the class Teams chat,
- ▶ during a lecture, submit to the vevox Q&A board,
- ▶ come to my student hour:
 - ▶ Tuesdays 1.00pm – 2.00pm, Bragg foyer,
- ▶ attend a **Maths Lunch Hour** (no lunch served, sorry)
 - ▶ Mondays and Wednesdays, 1.00pm – 2.00pm, Bragg GR.18.

Passive learning and active learning

Examples of passive learning:

- ▶ relying only on lecture recordings,
- ▶ relying only on youtube tutorials,
- ▶ relying only on AI summaries and AI assistance,
- ▶ not attempting any exercises (beyond the ones you are required to submit)

Examples of active learning:

- ▶ participating in lectures with questions and/or answers,
- ▶ reviewing lecture materials before and/or after class,
- ▶ attempting all worksheet exercises before the tutorial class.

Active participation is essential to learning.

Some study advice

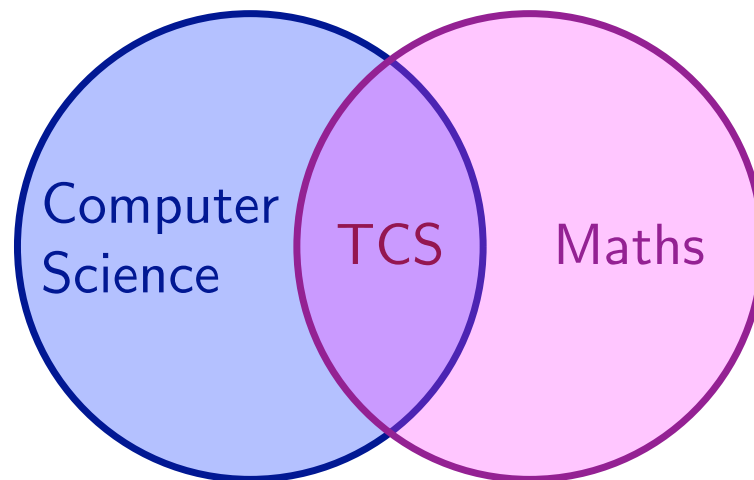
- ▶ Learning is never easy — we have to put in hard work to receive benefits.
- ▶ Mathematics and computer science are hard for everyone.
- ▶ Remember that you don't need to perfectly complete an exercise in order to learn from it — it's not about solving the problem, it's about working the problem.
- ▶ If you are entirely relying on outside assistance (such as AI assistance), you are cheating yourself out of your education.
- ▶ If all you are doing is listening and submitting portfolio exercises, you are **not** preparing for the exams.

Lecture behaviour

- ▶ If you arrive late to a lecture, please enter quietly and discreetly.
- ▶ The lecture theatre needs to be a place of quiet concentration — so that everyone has a chance to do their best learning.
- ▶ Chatter is distracting to everyone in the class, including the lecturer.
- ▶ You will be asked to leave the lecture theatre if you continue to create a distraction.

COMP1870: Theoretical Foundations of Computer Science I

What is Theoretical Computer Science?



Why (should a computer scientist) study mathematics?

- ▶ Because the university requires it.
- ▶ Because the accrediting bodies require it.
- ▶ Because mathematics helps us to find good algorithms.
- ▶ Because we need mathematics to prove that our algorithms behave as we expect.
- ▶ Because mathematics lets us analyse how good our algorithms are.
- ▶ Because mathematics lets us explore the limits of what computers can accomplish.

Mathematics provides a foundation for computer science.

Also, mathematics can be fun and beautiful (as well as hard and frustrating)!

The maximum-sized matching problem

Say that we have a collection of tasks we want to carry out using a collection of servers.

- ▶ Each server will carry out at most one task.
- ▶ Each server is able to carry out only some of the tasks.

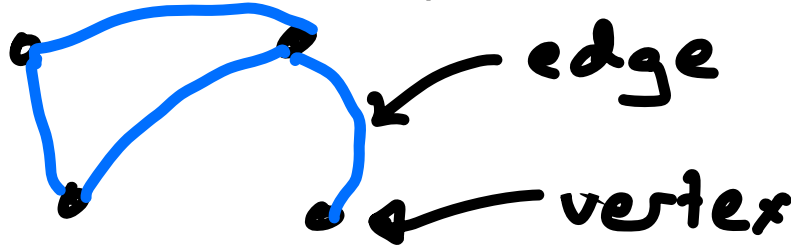
Server	can carry out tasks:
1	1,4,6
2	1,4,7
3	2,5,6
4	1,3,7
5	2,3,7

How many tasks can be assigned to servers?

Graphs

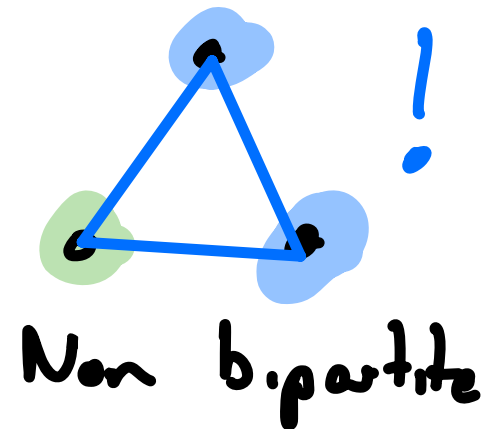
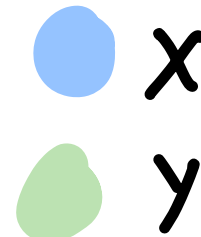
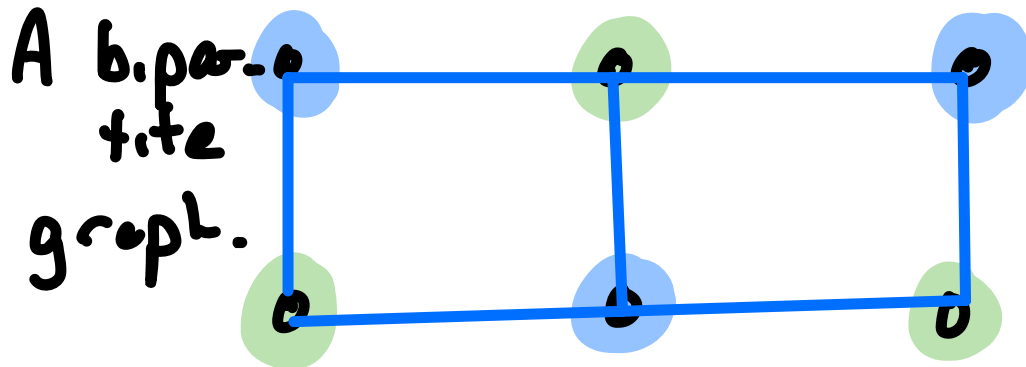
Every **graph** consists of:

- ▶ A **set** of vertices (represented by dots)
- ▶ A **set** of **edges** (represented by curves or lines that join pairs of vertices)



A picture of
a graph.

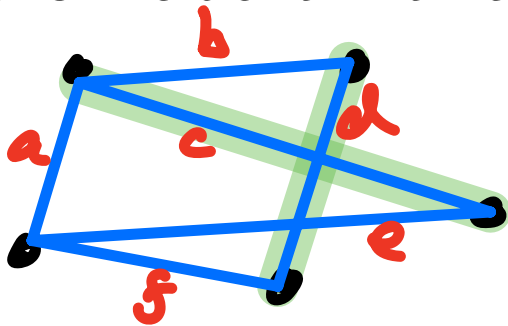
If the vertices can be divided into two sets, X and Y , in such a way that every edge joins a vertex in X to a vertex in Y , then the graph is **bipartite**.



Matchings

If an edge e joins vertices x and y , we say that x and y are **incident** with e .

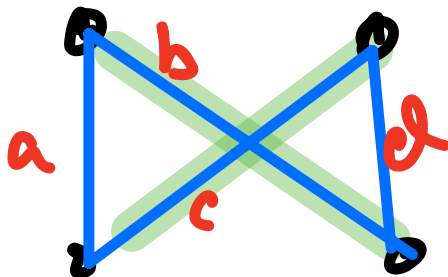
A **matching** is a set of edges such that every vertex is incident with at most one edge in the matching. A vertex is **matched** if it is incident with an edge in the matching.



Examples
of
matching

c, d
 a, d
 f

A **perfect matching** is a set of edges such that every vertex is incident with exactly one vertex in the matching.

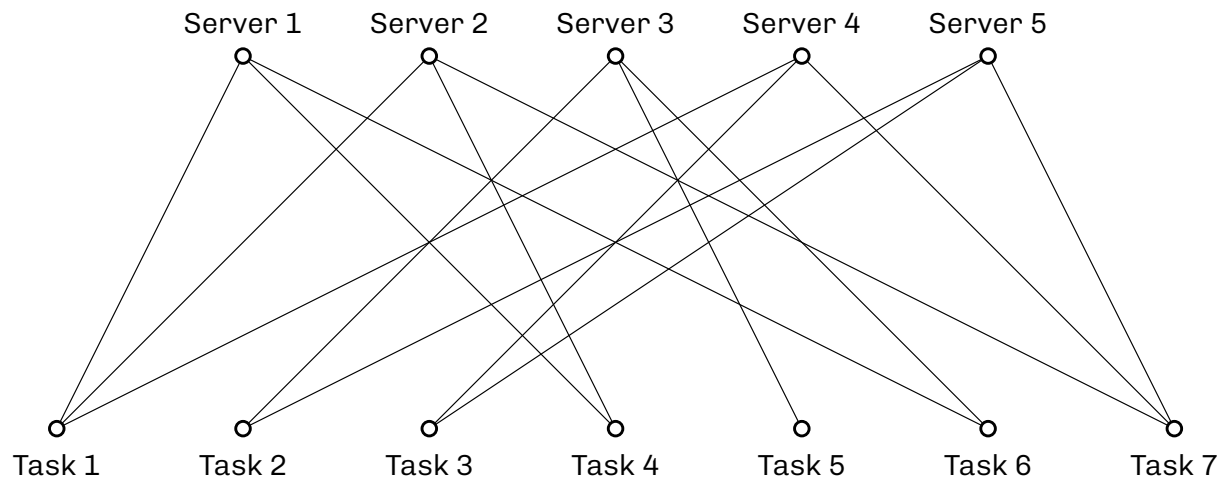


b, c is a perfect matching
 d is a matching, not perfect.

Matchings

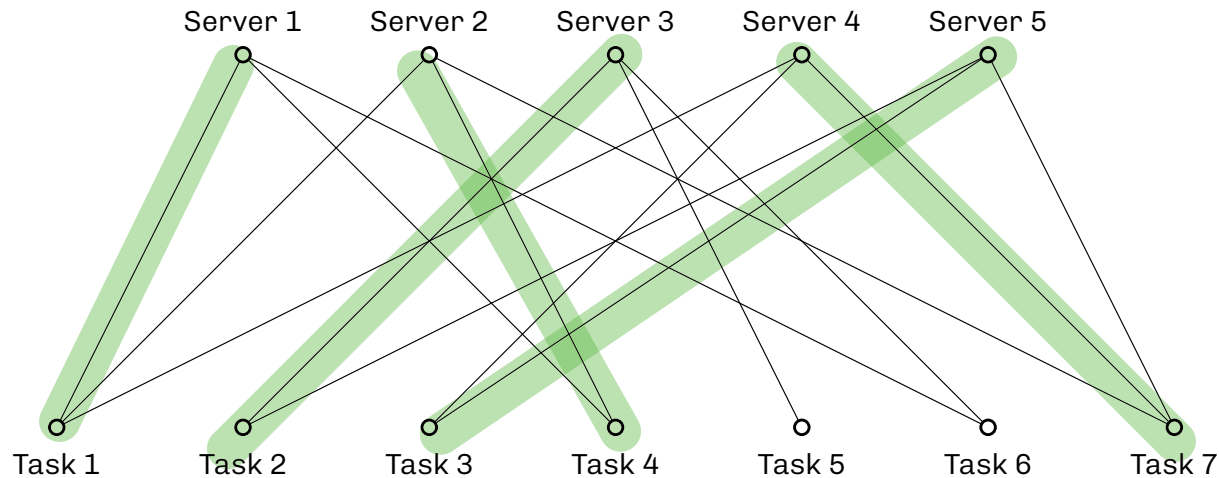
Server	can carry out tasks:
1	1,4,6
2	1,4,7
3	2,5,6
4	1,3,7
5	2,3,7

This situation can be represented as a bipartite graph:



Matchings

Each server can be assigned at most one task and each task is assigned to at most one server. So an assignment is a matching in the corresponding bipartite graph.



The matching consists of the edges that join a task to the server it has been assigned to.

Our problem is: find the matching with the maximum number of edges.

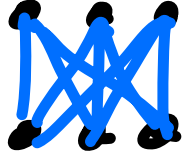
A naive approach

- ▶ Check every set of edges to see whether it is a matching.
- ▶ Keep a list of all matchings.
- ▶ Check the largest possible size of a matching.

How good is the naive approach?

Say that the bipartite graph has $|V|$ vertices.

What is the maximum possible number of edges?

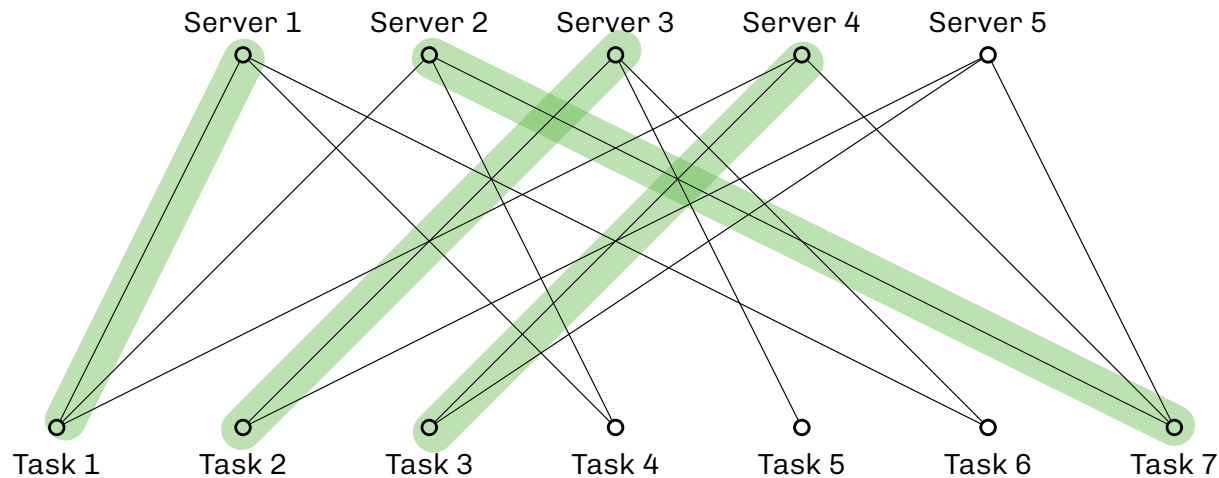

$$\left(\frac{1}{2}|V|\right) \times \left(\frac{1}{2}|V|\right) = \frac{1}{4}|V|^2$$

What is the maximum possible number of sets of edges?

$$2^{\left(\frac{1}{4}|V|^2\right)}$$

The greedy approach

- ▶ We build a matching edge by edge.
- ▶ We consider each edge in turn, in some random order.
- ▶ If we can add the edge to our matching and still have a matching, then we add it.
- ▶ If adding the edge would stop our matching from being a matching, we reject it.



This is not maximum sized.

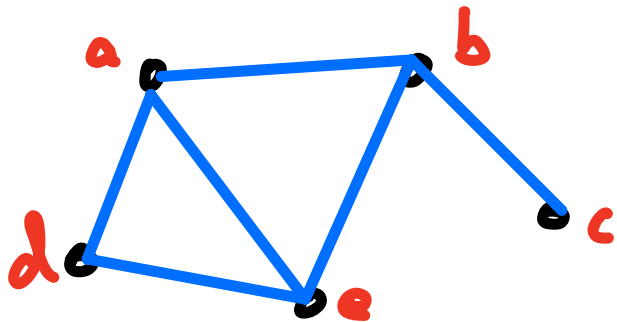
The current situation

- ▶ The algorithm which checks every possible matching is impossibly slow, unless we are using it only for very small examples.
- ▶ The greedy algorithm is fast, but it doesn't always produce a largest-possible matching.

There is a fast algorithm which always produces a largest-possible matching. We need some further mathematical ideas.

Alternating paths

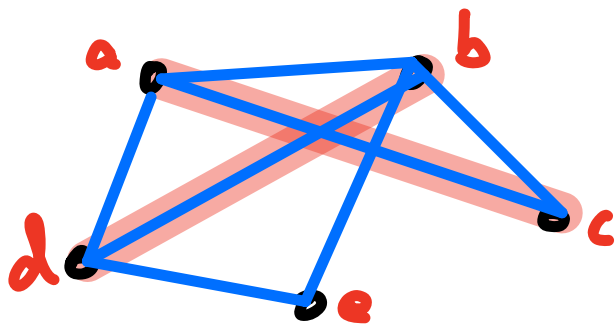
A **path** is a sequence of vertices with no repetitions, where each consecutive pair is joined by an edge.



Examples
of
paths:

a, b, c
 a, b, e, d
 e

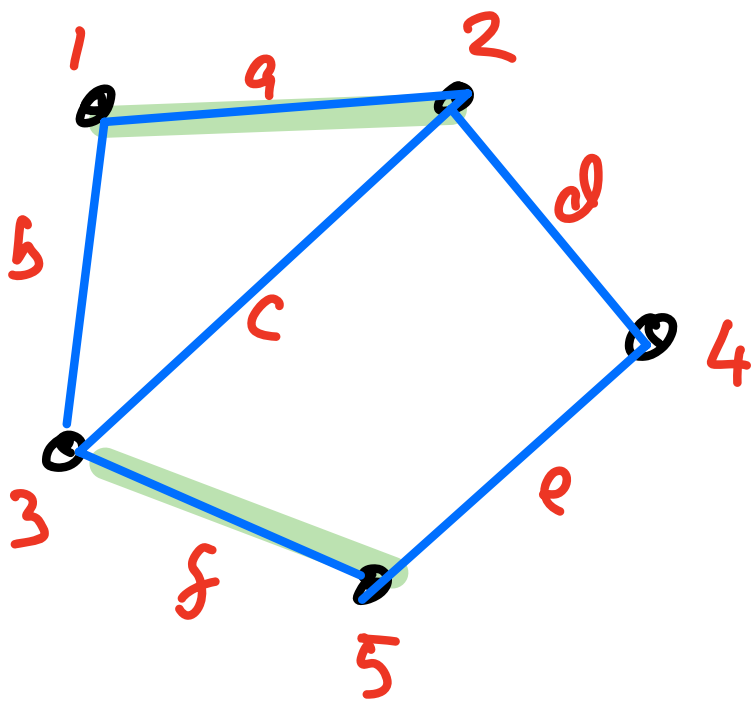
Assume that M is a matching in a graph. A path is **alternating** if the edges in the path alternate between being in the matching and not being in the matching.



The matching
consists of edges
 ac and bd .

Examples
of
alternating
paths:

b, d, e
 a, b, d
 a, b, d, e
 $e, d, b, c,$
 b, c, a, d



c, d, a
Not a matching
a, f

A matching is a set of some (maybe all) of the edges. No vertex is allowed incident with two or more edges in the matching.